

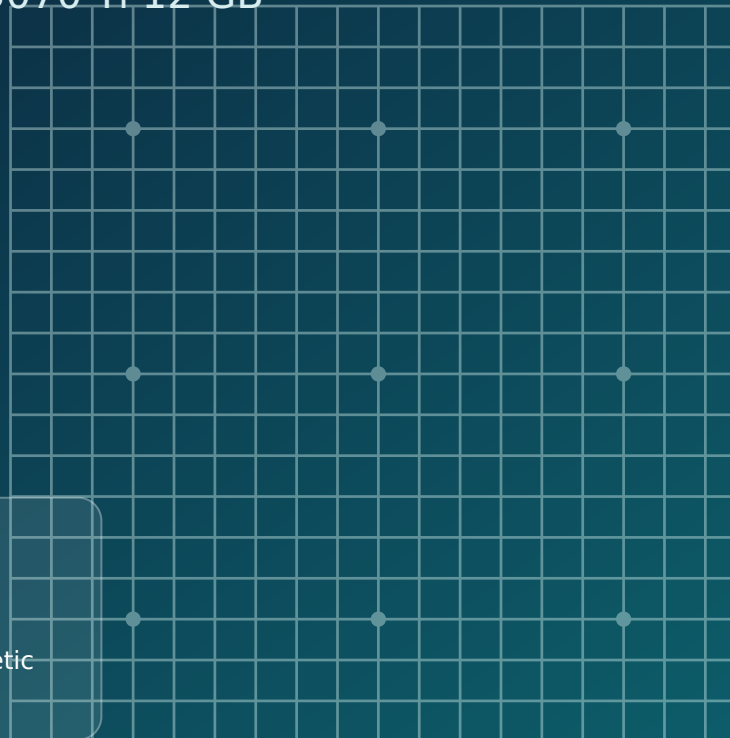
GTS-19: an exactness-first game-theoretic attempt for unrestricted 19×19 Go

A pinned mathematical game, collision-safe
superko identity, proof-number architecture, GPU/
host/NVMe execution plan, validation evidence, and
Codex work package for an RTX 5070 Ti 12 GB
laptop target.

EMPTY 19×19 ROOT

UNKNOWN

A bounded attempt was executed. No game-theoretic
winner is claimed.



Document control

DOCUMENT UGTS-KC 4.3 GTS-19 foundational report	PROOF TARGET 19×19 area · PSK · 7.5 komi
HARDWARE TARGET 12 GB VRAM user-specified laptop edition	ROOT STATUS UNKNOWN claim gate remains closed

Claim boundary

This release does not solve unrestricted 19×19 Go. It defines the exact target, proves the target is finite, supplies sound proof semantics and implementation scaffolding, validates miniature exact results, and records a bounded 19×19 attempt whose status remains **UNKNOWN**.

What “unrestricted” means here

The mathematical game has no artificial move cap, depth cap, or opening restriction. Practical executions may be bounded by nodes, time, RAM, VRAM, temperature, or disk. Such a stop is operational, not semantic: it returns `UNKNOWN`.

Version relationship

KC 4.3 is additive to the KC 4.2 Go solver baseline [P8]. It preserves deterministic rules, exact miniature search, serialized state, and certificates, then adds a pinned 19×19 proof target, threshold proof-number formulation, full-history symmetry constraints, compact GPU layouts, durable storage design, laptop memory policy, claim gates, and a Codex milestone ledger.

Contents

1 · Executive determination	3
2 · Exact game and rules profile	5
3 · Game-theoretic formulation	6
4 · UGTS operator order	7
5 · Exact identity, superko, and symmetry	8
6 · Search architecture	9
7 · RTX-laptop execution design	11
8 · Storage, checkpoints, and certificates	12
9 · Validation evidence	13
10 · Codex implementation programme	15
11 · Failure modes and claim controls	16
12 · Package map and reproduction	17
Appendix · Source integration register	20

01

Executive determination

The problem is mathematically precise and finite. The delivered implementation is a serious proof-search foundation, but the full 19×19 value remains open in this release.

Executive determination

UGTS-KC 4.3 converts “solve Go” into an auditable sequence of exact propositions. It separates what is proven, what is merely implemented, what is measured, and what remains unknown.

TINY FIXTURE

Black 0.5

empty 2×2, exact

OPENING CLASSES

56

55 placements + pass

19×19 ATTEMPT

UNKNOWN

threshold score₂ ≥ 1

What has been established

- ✓ A canonical 19×19 game profile is fully serialized.
- ✓ The game is finite under positional superko and two-pass termination.
- ✓ Half-point integer utility makes outcome and threshold arithmetic exact.
- ✓ Win/loss is represented as an AND/OR proof proposition.
- ✓ State identity includes the complete repetition context.
- ✓ D4 reduction is defined over board and history together.
- ✓ The retained exact 2×2 KC 4.2 regression result is rechecked from the preserved baseline certificate.
- ✓ The fast Python regression subset and dependency-free C++ CPU smoke build pass in the captured evidence.

What has not been established

- ✗ No threshold proof for the empty 19×19 root has completed.
- ✗ No complete CUDA legal-expansion kernel has been validated.
- ✗ No production persistent superko set or NVMe transposition database is finished.
- ✗ No standalone 19×19 strategy/proof DAG exists.
- ✗ No winner, exact margin, practical solve time, or storage requirement is known.

Useful outcome of the attempt

The release replaces an unbounded aspiration with an exact target, a falsifiable claim protocol, working reference code, hardware-aware interfaces, and a staged implementation path. That is meaningful progress without pretending the root was solved.

2 · Exact game and rules profile

“Standard Go” is not precise enough for a proof. A different scoring system, komi, repetition rule, suicide policy, or end protocol defines a different game. KC 4.3 therefore pins one target.

Field	Canonical value	Reason for serialization
Board	19×19, empty	Defines topology and root
First player	Black	Part of game state
Scoring	Area	Deterministic terminal utility
Komi	7.5 (komi2=15)	Avoids draws and floating point
Repetition	Positional superko	Makes history semantically relevant
Suicide	Illegal	Changes legal actions and values
End	Two consecutive passes	Explicit terminal condition
Move/depth cap	None in game definition	Unrestricted target

Profile identity

```
UGTS-G019-AREA-PSK-K7.5-v1
configs/go19_canonical.toml
objective: Black can force final score2 ≥ 1
```

The rules object travels with every checkpoint, certificate, benchmark, and result. A run with 6.5 komi or situational superko is not a continuation of this proof unless it is used only for non-authoritative experimentation.

Terminal score

$$U_2(S) = 2(A_B - A_W) - 15$$

An empty region counts for one player only when all bordering stones belong to that player; mixed or unbordered regions are neutral. The score is an odd integer in half-point units, so zero cannot occur.

No post-game dead-stone negotiation

The canonical proof target ends and scores mechanically after two passes. Introducing human adjudication would make the terminal function underspecified; a different protocol would need a new profile.

3 · Game-theoretic formulation

State and minimax value

$$S = (B, p, r, H, B_{-1})$$

B is the 361-point board; p is the player to act; r is the pass counter; H is the exact set of prior boards used by positional superko; and B_{-1} is retained for lineage/cross-profile audit.

$$V(S) = \max_{a \in L(S)} V(T(S,a)) \text{ for Black; } \min_{a \in L(S)} V(T(S,a)) \text{ for White}$$

Terminal states use U_2 . This recursion defines the game-theoretic score. It says nothing about the cost of computing it.

Why the unrestricted game is finite

Every non-pass move under positional superko must produce a board absent from the exact history. At most 3^{361} board colorings exist. A single pass may occur between non-pass moves, but a second consecutive pass ends the game. Therefore no legal play can be infinite.

COLORINGS

3³⁶¹

$\approx 10^{172.2}$

SCORE VALUES

723

odd values $-737 \dots 707$

THRESHOLD QUESTIONS

≤ 10

to isolate exact margin

These are logical bounds. They are not tractability estimates and do not imply the laptop can enumerate the state space.

Threshold propositions

$$W_t(S) \equiv \text{"Black can force } U_2 \geq t \text{ from } S"$$

At a Black node, the proposition is an OR over legal children. At a White node, it is an AND. For the win/loss question, $t=1$. Proof-number search can focus on the most proving frontier while retaining a binary truth condition.

Node	Proof number	Disproof number
Terminal true	0	∞
Terminal false	∞	0
Unknown leaf	1	1
OR (Black)	min child pn	sum child dn
AND (White)	sum child pn	min child dn

4 · UGTS operator order

The substrate contribution is not a decorative vocabulary. It forces the transition to be a sequence of explicit queries, guards, commits, witnesses, and replayable proof updates [P1-P7].



Point-move semantics

1. Query an empty point and establish board-topology support.
2. Propose placement without mutating the committed state.
3. Compute all adjacent opponent components.
4. Remove components with no liberties.
5. Recompute the placed stone's component after capture.
6. Reject forbidden suicide.
7. Reject a resulting board in the positional-superko set.
8. Atomically commit board, player, pass reset, history insertion, and lineage.
9. Emit deterministic content/state witnesses.
10. Update proof/disproof data only after every guard passes.

Why order is semantic

Checking own liberties before removing captured opponents rejects legal captures. Checking superko against a provisional pre-capture board tests the wrong state. Updating a proof number before durable child identity is confirmed can corrupt an entire certificate. The operator sequence is therefore versioned alongside the rules.

Atomicity invariant

A move either fails without changing state or commits a complete, replayable successor. GPU work may be speculative; the proof graph sees only verified commits.

Pass semantics

Pass preserves the board, swaps the player, and increments the consecutive-pass count. It is explicitly exempt from positional board repetition. Two passes produce a terminal state. Any non-pass move resets the counter.

5 · Exact identity, superko, and symmetry

Board equality is not state equality

Under superko, two nodes with the same current stones can have different legal futures because their prior-board sets differ. A board hash plus player is therefore an unsound transposition key. KC 4.3's Python oracle keys on board, player, pass count, previous board, and the complete sorted history set.

Forbidden optimization

Never reuse an exact proof value merely because current-board hashes match. A hash may select a candidate record; collision-safe complete identity must confirm it.

Persistent production identity

Copying all of H into each 19×19 node is impractical. The production design uses a persistent content-addressed set: a child history root is derived from the parent root and the inserted exact board object. GPU and host hash tables cache handles, while immutable NVMe segments retain collision-auditable bytes.

D4 symmetry

Rotation/reflection is sound only when applied to the current board, every history board, and previous-board lineage. With that condition, the empty root's 361 placements collapse exactly to 55 D4 orbits. Pass adds one class, so the first action frontier has **56** canonical classes.

RAW FIRST ACTIONS

362

361 placements + pass

D4 EXACT CLASSES

56

validated by selftest

Compact board encoding

Two bits encode empty, Black, and White. A 361-point board uses 722 bits, rounded to **91 bytes**. For GPU operations, two six-word bitplanes use 96 bytes and make occupancy masks inexpensive. Metadata and history handles remain in separate arrays.

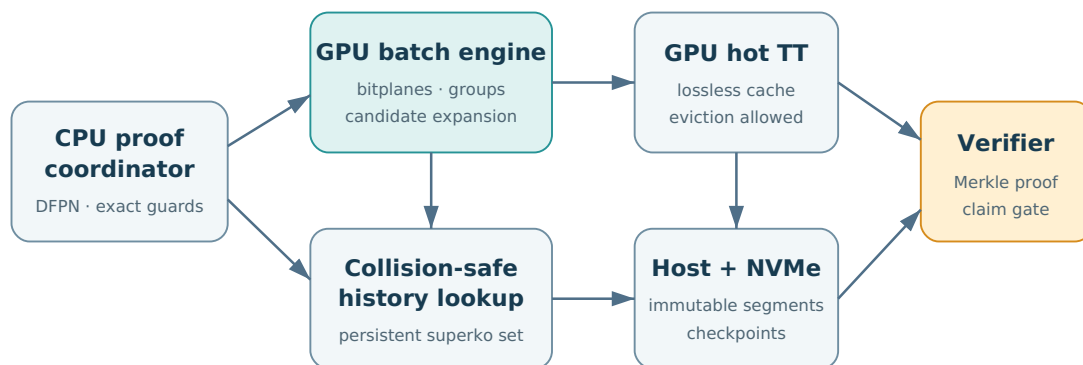
```
storage: 361 points × 2 bits = 722 bits = 91 bytes
gpu:     6 × uint64 black + 6 × uint64 white = 96 bytes
identity: compact board + exact history-root handle + collision audit
```

02

Search and execution

A sound proof coordinator can exploit GPU batches and storage hierarchy without allowing a heuristic or cache collision to decide truth.

6 · Search architecture



Reference and production roles

Component	Authority	Version 4.3 state
Python engine	Readable transition/scoring oracle	Implemented and tested
Python alpha-beta	Exact tiny-board regression	Implemented; not 19×19 scale
Python PNS	Transparent threshold semantics	Implemented; bounded explicit tree
C++17 engine	CPU production baseline	Transition/scoring smoke implementation
CUDA occupancy	Candidate acceleration only	Implemented scaffold; not full legality
Persistent history/TT	Production exact identity	Specified; implementation milestone
Standalone proof DAG	19×19 claim evidence	Specified; not implemented

Score-threshold campaign

The first campaign asks whether Black can force $\text{score}_2 \geq 1$. A proven root means a Black win; a disproven root means a White win. Only after that result is independently checked should additional thresholds be solved to isolate the exact margin.

Move ordering versus proof

Captures, center proximity, neural policy, tactical patterns, or prior statistics may decide which child is examined first. They cannot remove a child from an AND quantifier, declare a terminal, set a proof number to zero, or produce an exact score bound without a checkable witness.

Transposition records

Hot records store proof/disproof numbers, expansion state, compact state/history handles, and child ranges. Cold immutable records retain exact board bytes, persistent-history nodes, lineage, terminal score witnesses, and certificate links. Evicting a GPU cache entry may slow the run but must not lose durable proof state.

7 · RTX-laptop execution design

The target is the user's RTX 5070 Ti laptop configuration with 12 GB nominal VRAM. KC 4.3 avoids architecture-number assumptions and sizes allocations from runtime free memory.

Allocation policy

Call `cudaMemGetInfo` after selecting the device. Preserve 18% of the reported free amount. The table below is an **example for 10 GiB free**, not a claim that 10 GiB will be available.

TT 46% frontier 23% batch 16% proof 8% heur. 7%		
Pool	Example size	Role
Free VRAM supplied to planner	10.000 GiB	Example only; target must query runtime
Safety reserve	1.800 GiB	Display/driver variance and allocation margin
Usable after reserve	8.200 GiB	Pool distributed below
GPU TT cache	3.772 GiB	Hot proof/state handles; eviction allowed
Frontier	1.886 GiB	Most-proving and expansion records
Batch workspace	1.312 GiB	Bitplanes, group/liberty/capture buffers
Proof staging	0.656 GiB	Deltas awaiting durable commit
Optional heuristic	0.574 GiB	Ordering only; reclaimable

CUDA implementation boundary

The delivered kernel forms exact empty-point masks from black/white bitplanes. This is the first, easily differential-tested stage. A production child batch must still perform group labeling, captures, own-liberty/suicide, exact superko, deterministic encoding, and reference comparison.

No GPU proof claim in this build

The artifact build host's evidence validates the CPU path. The CUDA files are source scaffolding for the target laptop and must be compiled and differentially tested there before becoming proof-authoritative.

Laptop-safe operation

- Adapt batch size after allocation failure or thermal throttling.
- Use watchdog-friendly kernels when the GPU drives the display.
- Checkpoint before suspend, driver update, or long battery operation.
- Keep optional neural ordering memory reclaimable.
- Record exact legal children and proof updates per second—not only raw candidates.

Expected bottleneck hierarchy

For an exact superko-aware campaign, state identity, irregular history membership, durable TT traffic, and proof graph growth are likely to dominate before simple occupancy arithmetic. The architecture therefore treats the GPU as one layer of a proof system rather than as a self-contained brute-force oracle.

8 · Storage, checkpoints, and certificates

Content-addressed history

```
history_root(parent) + exact inserted board
-> collision-checked persistent-set node
-> history_root(child)
-> immutable segment + SHA-256 manifest
```

GPU caches can use compact hashes, but every proof-changing hit must resolve to collision-safe content. Probabilistic filters may accelerate misses or trigger exact lookups; they cannot reject a legal move by themselves.

Checkpoint transaction

1. Stop selecting new batches.
2. Finish or discard uncommitted GPU work.
3. Flush proof deltas to a write-ahead log.
4. Seal immutable segment files.
5. Hash segments and write a canonical manifest.
6. Atomically replace the checkpoint pointer.
7. Reopen and sample-verify before resuming.

Two certificate levels

Recomputation certificate · delivered

Serializes root/rules/result/digests. An independent process reruns the tiny exact solver and confirms the value. Suitable for regression, not a giant standalone proof.

Strategy/proof DAG · required later

Must quantify every relevant child, validate terminal scores, preserve complete history identity, and verify from a clean implementation before a 19×19 solved claim.

Claim gate

`scripts/claim_gate.py` rejects an unfinished result. Even a root proof number of zero is not publishable in 4.3 without the full certificate format and independent verifier. The package intentionally makes overclaiming harder than printing a search log.

03

Evidence and implementation programme

Every reported success below comes from captured package evidence. The 19×19 run is explicitly bounded and remains unknown.

9 · Validation evidence

FAST PYTHON TESTS

17/17

635.0 ms · exhaustive 2×2 excluded

C++ CPU

PASS

configure · build · smoke

19×19 ROOT

UNKNOWN

bounded PNS attempt

Retained exact empty 2×2 regression

Measure	Captured value
Status	EXACT
Game-theoretic result	Black by 0.5 points
Half-point utility	1
Best move	A2
Nodes	742
Terminal nodes	196
Alpha-beta cutoffs	343
TT entries	538
Maximum ply	38
Certificate verification	EXACT recomputed and digest-checked

This miniature value, certificate hash, and search metrics come from the validated KC 4.2 baseline preserved inside the 4.3 package. They are used as a regression target because the report rebuild did not rerun the slower 4.3 exhaustive 2×2 implementation.

Bounded empty 19×19 attempt

Measure	Captured value
Status	UNKNOWN
Threshold	Black score ₂ ≥ 1
Expanded nodes	64
Generated nodes	23,107
Proof number	1
Disproof number	361
Maximum explored ply	2
Elapsed	316.7 ms

Interpretation

Both proof and disproof numbers remained nonzero when the node budget ended. The only valid conclusion is **UNKNOWN**. The run validates transition, score-threshold, and status plumbing; it does not indicate which player wins.

C++ smoke evidence

Configure/build/smoke exit codes were 0/0/0. The smoke executable reported 26 legal initial actions on 5×5 including pass and produced packed bitplanes and a deterministic diagnostic digest.

10 · Codex implementation programme

Codex receives a repository-level contract, a ready-to-paste prompt, executable acceptance gates, and ordered milestones. The first target is semantic parity—not an unverifiable performance rewrite.

Read/run order

1. `codex/AGENTS.md`
2. `docs/EXACTNESS_CONTRACT.md`
3. `docs/FORMAL_SPEC.md`
4. `docs/GPU_ARCHITECTURE.md`
5. `codex/TASKS.md`
6. `codex/acceptance.sh` or `acceptance.ps1`

Milestone ladder

Milestone	Deliverable	Exit gate
M0	Reproduce Python/C++ evidence on target	All baseline gates pass
M1	C++ transition/scoring parity	$\geq 1,000,000$ differential transitions, zero mismatches
M2	Persistent exact history + NVMe checkpoint	Restart/collision-injection audit passes
M3	Production DFPN coordinator	Fixture values/certificates match
M4	Full exact CUDA expansion	10,000,000 child comparisons, zero mismatches
M5	Proof-safe reductions	On/off equivalence and witnesses
M6	Progressive board-size campaign	Independent exact verification
M7	Empty 19×19 threshold 1	Root pn=0 or dn=0 plus certificate
M8	Exact score margin	One of 723 scores isolated

First Codex assignment

Implement a deterministic cross-language trace format and differential harness. Compare legal actions, captures, exact board transitions, pass state, positional-superko rejection, terminal recognition, and area score between Python and C++. Minimize every mismatch into a permanent fixture. Do not begin proof-authoritative CUDA work before parity passes.

Why this ordering matters

A fast legality kernel with one rare ko or capture error can produce a compact, internally consistent, completely false proof. Differential tests and collision injection are therefore foundational performance work: they protect every later optimization.

11 · Failure modes and claim controls

Failure mode	Consequence	Control
Board-only TT key	Illegal superko transposition	Complete history identity; injected-collision tests
Hash collision treated as equality	False legal/repetition/proof value	Exact content comparison and immutable bytes
GPU race changes child order/content	Non-reproducible or missing children	Deterministic encoding and CPU differential audit
Proof counter wraps	False zero or bound	Saturating declared-width arithmetic
Neural estimate stored as bound	Heuristic becomes false proof	Ordering-only type separation
Incomplete checkpoint	Lost or inconsistent proof graph	WAL, immutable segments, atomic pointer, hashes
Rules drift	Solves a different game	Profile/root digest on every artifact
Budget stop mislabeled final	Overclaim	UNKNOWN status and claim gate
Local decomposition ignores ko threats	Unsound pruning	Certified interfaces and external-threat accounting
Laptop thermal/driver interruption	Corruption or lost work	short kernels, adaptive batches, frequent checkpoints

Publication checklist for a future solved claim

- ✓ Canonical rules and empty-root digests match.
- ✓ Root proof or disproof number is exactly zero.
- ✓ Every quantified child and terminal score is represented or independently derivable.
- ✓ Complete repetition identity is verifiable.
- ✓ All segment and Merkle hashes verify.
- ✓ A clean independent implementation accepts the certificate.
- ✓ Result is reproduced after checkpoint restore.

✓ Remaining assumptions and hardware faults are disclosed.

Current gate state

Closed. The package's bounded root run is UNKNOWN and the standalone 19×19 proof verifier is not yet implemented.

12 · Package map and reproduction

Core directories

README.md	src/go_state.cpp
AGENTS.md	cuda/packed_kernels.cu
pyproject.toml	cuda/gpu_probe.cu
src/ugts_go19/	configs/
engine.py	schemas/
exact.py	docs/
pns.py	codex/
symmetry.py	scripts/
codec.py	tests/
certificate.py	fixtures/
cpp/	evidence/
CMakeLists.txt	baseline/

Linux/macOS reproduction

```
python -m venv .venv
source .venv/bin/activate
python -m pip install -e .
./codex/acceptance.sh
```

Windows PowerShell reproduction

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -e .
.\codex\acceptance.ps1
```

Target-laptop GPU probe

```
python scripts/hardware_probe.py evidence/local_hardware.json
cmake -S cpp -B build-cuda -DUGTS_ENABLE_CUDA=ON `
  -DCMAKE_BUILD_TYPE=Release -DCMAKE_CUDA_ARCHITECTURES=native
cmake --build build-cuda --config Release
```

Integrity

SHA256SUMS.txt covers release files except itself and transient build caches. scripts/verify_release.py checks the manifest. The external PDF and ZIP hashes are supplied with delivery because a file cannot reliably embed its own final hash.

Recommended first target run

Run M0 and M1 only. Capture hardware and compiler versions, build both references, regenerate tiny certificates, then implement the cross-language differential harness. Do not launch a long 19×19 campaign until exact history/checkpoint and full CUDA legality gates exist.

Conclusion

KC 4.3 does not turn a 12 GB laptop into a magic exhaustive oracle. It turns the request into a precise, finite, testable proof programme whose failures are visible and whose eventual success would be independently checkable.

The strongest current statements are:

- the canonical unrestricted game is well-defined and finite;
- the first-action symmetry reduction is exact;
- miniature exact search and certificates work;
- the CPU reference path passes captured validation;
- the GPU, storage, and proof interfaces are specified for Codex;
- the empty 19×19 root remains **UNKNOWN**.

Foundation delivered

The accompanying ZIP is designed to be opened directly as a Codex repository. Its acceptance gates preserve correctness while Codex ports semantics to C++, completes exact CUDA expansion, implements durable history/TT storage, and advances the progressive proof campaign.

Upgrade identifier

```
UGTS-KC 4.3.0 · GTS-19
proof profile: UGTS-G019-AREA-PSK-K7.5-v1
hardware target: RTX 5070 Ti Laptop · 12 GB VRAM (user specified)
root outcome: UNKNOWN
```

Appendix · Source integration register

The following project documents were supplied as the substrate lineage. KC 4.3 uses them as conceptual/provenance inputs and preserves the prior Go package under `baseline/` when available. No unrelated wallet or spatial-scanning semantics enter the Go rules.

ID	Artifact	Integrated role
P1	UGTS_KC_2_0_Tom_Klootwijk_Expanded_Substrate.pdf	state/query/event and lineage foundation
P2	UGTS_KC_Two_Hands_3_0_Interactive_Graphics_Runtime.pdf	proposal, guard, and deterministic commit discipline
P3	UGTS_KC_3_6_Tom_Klootwijk.pdf	definition-addressed reproducibility
P4	UGTS_KC_3_6_2_SCLP_Tom_Klootwijk.pdf	finite packing and explicit guard/failure boundaries
P5	UGTS_KC_3_9_KC_Elizabeth_Vector_Game_Runtime.pdf	deterministic game-runtime lineage
P6	Unified_Geometric_Topological_Substrate_GPU_Native_Addendum.pdf	GPU-native state and evidence boundary
P7	UGTS_KC_4_2_General_Operator_Order_Addendum(1).pdf	explicit operator and order serialization
P8	UGTS_KC_4_2_GO_Solver_Report.pdf	prior Go rules/search/certificate baseline
P9	UGTS_Versioning_Charter_Phase_1_Foundation_Chronology(1).pdf	version-lineage discipline

Normative files in the 4.3 package

File	Normative scope
<code>configs/go19_canonical.toml</code>	Rules and proof objective
<code>docs/FORMAL_SPEC.md</code>	State, transition, utility, minimax, threshold semantics
<code>docs/EXACTNESS_CONTRACT.md</code>	Allowed and forbidden optimizations
<code>schemas/*.json</code>	Certificate and checkpoint serialization
<code>codex/AGENTS.md</code>	Agent invariant and claim discipline

Generated from captured release evidence on 29 August 2026. Floating-point values in timing tables are measurements of this artifact build only; they are not predictions for the target laptop.